

백엔드 포트폴리오

Spring Boot 기반 온라인 저지 서비스 개발 과정에서 정리한 포트폴리오 문서를 하나로 합친 문서다.

목차

1. 대회 제출 Insert 경로 최적화
2. JUDGE 서버 설계 과정
3. Redis Scoreboard Recovery
4. Rank 조회 경로 최적화

1. 대회 제출 Insert 경로 최적화

핵심 아이디어

핵심 아이디어는 MySQL의 batch rewrite를 실제로 활용하는 것이었다.

JDBC batch와 `rewriteBatchedStatements=true`가 제대로 동작하면 여러 insert가 아래처럼 하나의 multi-values insert로 합쳐질 수 있다.

```
insert into contest_submission (...) values (...), (...), (...);
```

어떻게 구현했는가

요청을 바로 insert하지 않고 잠시 큐에 모은 뒤 chunk 단위로 flush하도록 구성했다.

큐에서 꺼낸 요청을 하나의 트랜잭션, `EntityManager` 안에서 처리하여, Hibernate batch와 MySQL rewrite가 적용되도록 만들었다.

- ID를 DB auto increment가 아니라 애플리케이션에서 먼저 할당했다
- `saveAll(...)`을 사용하면, JPA가 `merge` 경로를 타고 row마다 `select ... where id = ?`를 발생시킨다. `entityManager.persist(...)` 기반으로 바꾸면서 제거했다.

테스트 결과

시나리오	immediate	bulk
insert 1건, pool 100	첫 실패 4000 RPS	첫 실패 6000 RPS
insert 3건, pool 100	첫 실패 4000 RPS	첫 실패 5000 RPS
insert 1건, pool 10	첫 실패 2000 RPS	첫 실패 4000 RPS
insert 3건, pool 10	첫 실패 2000 RPS	첫 실패 4000 RPS

실패했던 부분

1. OSIV가 켜져 있으면 lazy로 `SELECT`가 숨어있기 쉽고 큐에서도 커넥션을 잡고 있어서 커넥션이 고갈됐다.

2. `saveAll(...)` 때문에 신규 insert인데도 `merge` 경로를 타면서 row마다 `select ... where id = ?` 가 붙고 있었다.

2. JUDGE 서버 설계 과정

배경

채점서버는 rest api로 채점을 지원한다.

최초에 대회 제출 후 처리는 event를 발행하여 asny로 건별로 api를 호출후 결과로 `contest_submission_result`, `contest_submission_outbox` 를 생성했다.

문제는 대회제출의 부하가 그대로 채점서버로 옮겨가고 spring 서버가 죽었을 때 복구가 안된다는 점이다.

1. 한 트랜잭션 구조

1. DB에서 `PENDING` 제출을 `SELECT ... FOR UPDATE SKIP LOCKED` 로 여러 건 조회한다
2. 같은 트랜잭션 안에서 judge를 병렬 실행한다
3. judge가 모두 끝나면 `contest_submission_result`, `contest_submission_outbox` 를 쓴다
4. 마지막으로 `contest_submission.judge_status = DONE` 으로 갱신하고 commit 한다

- contest submission때와는 달리 insert ignore이 필요해서 jdbc batch를 이용했다.
- 중간에 프로세스가 죽으면 트랜잭션 rollback으로 정리된다
- 채점서버 api 병렬 호출 수준을 조절 가능.

채점서버 가정 및 한계 : - 병렬로 10개씩 처리 가능하고 대회시에는 보통 제출이 10ms걸린다고 생각했다.(대회시 부분채점) - 하지만 간단한 검증을 가지고 있어도 코드를 잘못짤 수도 있기에 1000건당 하나만 2초정도 걸린다고 가정. - 지금 구조에서는 트랜잭션 하나에서 다른 judge의 완료를 기다려야 하기에 100건이 다 10ms면 100ms예상. - 하나라도 2초가 걸리는 게 있으면 그동안 judge의 병렬 처리를 전혀 이용하지 못한다.

2. 현재 구조

1. claim

DB에서 `PENDING` 제출을 짧은 트랜잭션으로 선점한다.

- `judge_status = 'PROCESSING'`
- `judge_claim_token = ?` : 다시 조회를 위해 필요
- `judge_claimed_at = now` :회복을 위해 필요

선점 후 queue에 넣는다.

2. judge

- worker가 queue에서 제출을 꺼내 채점서버 api 호출한다.

응답을 queue에 넣는다.

3. write

queue에서 일정 건씩 꺼내 batch로 한 번에 처리한다.

1. `contest_submission_result` insert
2. `contest_submission_outbox` insert
3. `contest_submission.judge_status = DONE` update

4. recovery

- `PROCESSING` 인데 `judge_claimed_at` 이 일정 시간보다 오래되면 다시 `PENDING` 으로 복구한다 채점 서버는 멍등성이 보장된다는 가정이다.

3. 한계

- recovery timeout 현재 채점 서버 api 호출시간이 최대 2초로 가정하고 있는데 이것을 준수해도 100건을 읽는다고 했을 때 어느 입력에서 100건이 모두 2초가 걸리는 것 같은 최악의 상황까지 가정하면 넉넉하게 잡아야하는데 그럴수록 회복이 늦어질 수 있다.
- 메모리 queue 구간은 durable하지 않다 각 과정의 병목에 서로 영향을 받지 않기위해 queue를 2개 두어서 분리했다. 이것은 중간 설계의 문제를 해결하지만 그만큼 durable하지 않은 구간을 늘린다.

둘 다 정합성보다 중복 처리 비용 문제에 가깝다.

3. Redis Scoreboard Recovery

무엇을 해결했는가

대회 스코어보드를 Redis에 두고 redis가 죽었을 때 효율적인 회복법을 구현했다.

score board 구조

judge 서버 : outbox 생성 batch 서버 : outbox 주기적으로 redis로 전송 redis score board : score board 관리

문제

복구할때 `contest_submission_outbox.id` 는 아쉬움이 있다.

- `outbox.id` 는 DB insert 순서이지 Redis 적용 순서가 아니다.
- 실패와 재시도로 인해 일부 id만 비어 있을 수 있다.

`outbox.id` 만으로는 Redis 회복이 어렵다.

비교

자연 복구

- Redis에서 하나의 처리를 완료할 때마다 `redis_seq` 를 발행하고 이를 outbox에 저장한다.
- 만약 같은 값이 들어오면 같은 `redis_seq` 를 반환한다.
- Redis가 죽었다가 RDB 방식으로 스냅샷을 복구하면 `redis_seq` 도 낮아져서 outbox에서 duplicate `redis_seq` 가 발생한다
- batch 서버가 이 중복 seq를 주기적으로 찾아 replay한다

장점: 별도 복구 모드가 필요 없어서 운영 흐름을 끊지 않는다

단점: 복구 속도가 트래픽에 영향을 받는다

대안 DB gapless seq

DB가 `gapless seq` 를 직접 발급한다.

- outbox insert 시점에 DB가 별도 seq를 부여한다
- Redis는 이 seq를 그대로 받아 scoreboard 반영과 복구 기준으로 사용한다

- 복구 시에는 Redis가 성공하지 못한 가장 작은 seq를 보내고, Spring에서 그 seq부터 이미 처리됐어야 할 것들을 다시 밀어 넣는다 (go back n)

장점: 트래픽이 복구에 필요하지 않다.

단점 : - gapless를 보장하려면 seq 발급 구간을 직렬화해야 한다 - 위의 방식과는 다르게 회복을 여러 서버에서 돌리기 쉽기 않다.

선택

자연복구 방식을 사용하고 대회가 끝나고 나서 혹시 있을지도 모르는 미처리 중복을 처리해준다.

4. Rank 조회 경로 최적화

무엇을 만들고 있었는가

랭킹 페이지를 만들 수 있는 기본적인 쿼리는 아래와 같을 것이다.

```
SELECT ...  
FROM user u  
WHERE ...  
ORDER BY score DESC, last_solved_date ASC, id ASC  
LIMIT :offset, :limit
```

병목의 핵심은 정렬 자체보다 **OFFSET**이며, 유저 수가 커질수록 뒤쪽 페이지 비용이 거의 선형으로 증가한다.

- **around me** 기능은 유저가 자신의 랭킹 페이지로 이동하는 기능이다. 한 번에 자기 위치로 이동하기 때문에 커서를 사용하기도 힘들다.

핵심 아이디어

1. solved count

solved_count_bucket 테이블에 아래 정보를 저장했다. - 풀 문제 수 **n** - 해당 solved count를 가진 유저 수 - 자신보다 solved count가 큰 유저 수

1. 내 solved count를 읽는다
2. bucket에서 나보다 solved count가 큰 유저 수를 읽는다
3. 같은 solved count를 가진 유저들 중에서 내가 몇 번째인지 계산한다
4. 둘을 더해서 내 rank를 구한다
5. 그 rank가 속한 page start를 계산한다
6. 해당 solved count 구간에서 page size만큼 읽는다

이 구조가 가능한 이유는 **solved_count** 가 1씩 증가만 할 수 있기 때문이다. 그래서 전체 랭킹을 매번 다시 정렬하지 않고, 작은 bucket 테이블만 유지해도 내 rank를 계산할 수 있다.

다만 이 방식의 성능은 전체 유저 수 만이 아니라 **값의 분포** 에 크게 좌우된다.

2. current streak

현재 스트릭은 하루 한 번 배치 시점에 0이 아닌 유저만 모은 `user_streak_rank_snapshot` 을 생성한다.

1. snapshot에서 내 row를 바로 찾는다
2. 거기에 이미 저장된 `snapshot_rank` 를 읽는다
3. 그 rank로 page start를 계산한다
4. `snapshot_rank BETWEEN start AND end` 로 해당 page만 읽는다

current streak는 조회 시점에 내 rank를 계산 하는 것이 아니라, 이미 계산된 rank를 읽는 구조다.

테스트 결과

비교는 각 랭크 타입마다 앞/중간/뒤 지점에서 수행했다.

Rank type	Front	Middle	Back
current streak naive	161 ms	8,890 ms	14,382 ms
current streak optimized	0.586 ms	0.443 ms	0.113 ms
solved naive	19.1 ms	217,842 ms	233,231 ms
solved optimized	24.2 ms	369 ms	47,747 ms

10만 유저 데이터셋에서 같은 solved bucket 방식의 deep page fetch는 약 0.46 ms 수준이었다.